# Tasks

## 1. Explain the steps taken to calculate "trfm_school_board_reports_long".

Within the object corresponding to "trfm_school_board_reports_long", the "dependencySourcings" attribute defines which tables are used to calculate the result of this transformation process.

Thus, attempts = "trfm_ta_detail_in_sb" dataframe and item_responses = "trfm_responses_with_domain" dataframe.

The steps taken can be seen through the code below where the “kind” attribute describes which steps are taken. These are followed in order.

```
"sequence": [
      {
        "kind": "join",
        "df_output": "result",
        "config": {
          "how": "inner",
          "left": "attempts",
          "right": "item_responses",
          "left_on": "attempt_id",
          "right_on": "ta_id"
        }
      },
      {
        "kind": "restrict-cols",
        "df_output": "result",
        "config": {
          "df_input": "result",
          "cols": [...]
        }
      }
    ],
```

These dataframes are then used with the steps described in "methodConfig" as follows:

1) The two dataframes described by attempts and item_responses variables respectively are inner joined together to form a larger dataframe called

“result”. This is done on the basis of matching the values in the columns “attempt_id” in the attempts dataframe (i.e. "trfm_ta_detail_in_sb" dataframe) and "ta_id" in the item_responses dataframe (i.e. "trfm_responses_with_domain" dataframe). The “result” dataframe then contains all the columns combined from both the dataframes with the rows having the same values for “attempt_id” and “ta_id”.

2) Here, taking the “result” dataframe generated in the previous step, specific column names are defined which should be retained because it had too many columns or even repeated ones from the join process. This new dataframe is reassigned to the “result” variable so that it is the final result of this transformation process.

## 2. What are the columns in that table, and where do they come from?

The columns in the table are shown below. They come from merging the two data frames through the join process in step 1) as described in Q1. Then since there can be repeats, the column restriction process specifies the columns to be retained which are listed below.

```
"cols": [
            "test_window_id",
            "tw_type_slug",
            "s_name",
            "sc_name",
            "uid",
            "stu_gov_id",
            "first_name",
            "last_name",
            "window_code",
            "lang",
            "item_id",
            "item_domain",
            "is_practice_test",
            "score",
            "weight",
            "is_nr",
            "formatted_response",
            "response_type"
          ]
```

## 3. What problems or inefficiencies are there in that calculation? How could it be improved?

1) If it was originally clear that there were specific columns to be retained from all the columns that exist, it would have been efficient to trim both the tables so that only the final columns to be retained plus “attempt_id” and “ta_id” are there. In this way, after the join process, the “result” variable won’t have to store a huge dataframe, but a smaller one.

2) One problem is that if two dataframes referred by attempts and item_responses aren’t sorted, the join process will become very inefficient. This is because when matching from left dataframe to right frame, the matching record / row could be at the end of the right table. This means there would be an O(N^2) like complexity, which isn’t ideal. To solve this, both the tables should be sorted such that all different values of “attempt_id” are sorted in ascending order. Then for a given “attempt_id”, the “ta_id” should be sorted in ascending order too. This way a binary search or another search algorithm could efficiently collect the records that have matching values for these two columns respectively.

## 4. Complete the JSON for the incomplete asset `trfm_scores_by_domain`.

Firstly, as the name starts with “trfm”, this object refers to a transform process. Thus, data columns must be extracted from other objects and then transformed so that the final columns of the returned data frame adhere to those in “schema_simple”.

So, firstly I found the slugs which have “trfm_scores_by_domain” as dependency. These were “trfm_sb_domain_accuracy_thresholds” and “trfm_sb_accuracy_per_item_domain”. This was done to make sure that to create “trfm_scores_by_domain”, we can neither use these processes as they themselves depend on “trfm_scores_by_domain” nor the processes which depend on those two.

Since "description": "Total scores for each item_domain for each attempt.", we can infer that each student, i.e. a specific “uid” , gives a test, which belongs to a particular item_domain and that particular test has a window code. Then within the test,  submitted attempts have an attempt_id, n_seen, and n_correct.

Also, tw =  test windows and ta = test attempts. This suggests that each user can make multiple attempts at the test of a particular test window. This is in the slug “load_ta_from_tw”.

From the dependencies of Trfm_school_board_reports_long, I can see that all the scoring information relating to a particular attempt (n_seen, n_correct, score, weight) must be retrieved from trfm_ta_detail_in_sb because, as the name suggests, it gives the detailed info about a test attempt with a given attempt_id.

And then, trfm_responses_with_domain, another dependency of Trfm_school_board_reports_long must have the information about the domain to which a particular test item (whose detailed response info is given in trfm_ta_detail_in_sb) belongs.

Thus, these are the only two dependencies which must be used. So first I put:

```
"dependencySourcings":[
    {
       "param": "attempts",
       "type": "dataframe",
       "config": {
         "asset": "trfm_ta_detail_in_sb"
       }
     },
     {
       "param": "item_responses",
       "type": "dataframe",
       "config": {
         "asset": "trfm_responses_with_domain"
       }
     }
    ]
```

Next, I know from Trfm_school_board_reports_long implementation that they share attempt_id (in “attempts” dataframe) and ta_id (in “item_responses” dataframe). So they must be inner joined through that column. Thus, the first step in the sequence must be:

```
{
         "kind": "join",
         "df_output": "joined_df",
         "config": {
           "how": "inner",
           "left": "attempts",
           "right": "item_responses",
           "left_on": "attempt_id",
           "right_on": "ta_id"
         }
       }
```

Next, since each student has a uid, who gives multiple attempts on items with different item_domains, we must have that each record has a unique uid, item_domain combination. Thus, I grouped the result (joined_df) of the previous process by “uid” and “item_domain”. All  the rows which matched a particular combination were then combined by summing their numerics as seen below to calculate totals.

```
        "kind": "group-by",
        "df_output": "result",
        "config": {
          "df_input": "joined_df",
          "group_by": [
            "uid",
            "item_domain"
          ],
          "agg": [
            {
              "col_new": "total_score",
              "agg_type": "sum",
              "col_target": "score"
            },
            {
              "col_new": "total_weight",
              "agg_type": "sum",
              "col_target": "weight"
            },
            {
              "col_new": "n_seen",
              "agg_type": "sum",
              "col_target": "n_seen"
            },
            {
              "col_new": "n_correct",
              "agg_type": "sum",
              "col_target": "n_correct"
            }
          ]
        }
      }
```

Finally, since we only need specific columns in the final dataframe which is returned by this method, I restrict columns to match them to the schema_simple:

```
{
        "kind": "restrict-cols",
        "df_output": "result",
        "config": {
          "df_input": "result",
          "cols": [
            "window_code",
            "uid",
            "item_domain",
            "total_score",
            "total_weight",
            "n_seen",
            "n_correct"
          ]
        }
      }
```

**Full JSON Code:**

```
{
   "slug": "trfm_scores_by_domain",
   "description": "Total scores for each item_domain for each attempt.",
   "structure": "dataframe",
   "method":"transforms",
   "schema_simple": [
     "window_code",
     "uid",
     "item_domain",
     "total_score",
     "total_weight",
     "n_seen",
     "n_correct"
   ],
    "methodConfig": {
     "sequence":[
       {
         "kind": "join",
         "df_output": "joined_df",
         "config": {
           "how": "inner",
           "left": "attempts",
           "right": "item_responses",
```

```
        "left_on": "attempt_id",
        "right_on": "ta_id"
      }
    },
    {
      "kind": "group-by",
      "df_output": "result",
      "config": {
        "df_input": "joined_df",
        "group_by": [
          "uid",
          "item_domain"
        ],
        "agg": [
          {
            "col_new": "total_score",
            "agg_type": "sum",
            "col_target": "score"
          },
          {
            "col_new": "total_weight",
            "agg_type": "sum",
            "col_target": "weight"
          },
          {
            "col_new": "n_seen",
            "agg_type": "sum",
            "col_target": "n_seen"
          },
          {
            "col_new": "n_correct",
            "agg_type": "sum",
            "col_target": "n_correct"
          }
        ]
      }
    },
    {
      "kind": "restrict-cols",
      "df_output": "result",
      "config": {
        "df_input": "result",
        "cols": [
          "window_code",
          "uid",
```

```
            "item_domain",
            "total_score",
            "total_weight",
            "n_seen",
            "n_correct"
          ]
        }
      }
    ]
  },
  "dependencySourcings":[
  {
      "param": "attempts",
      "type": "dataframe",
      "config": {
        "asset": "trfm_ta_detail_in_sb"
      }
    },
    {
      "param": "item_responses",
      "type": "dataframe",
      "config": {
        "asset": "trfm_responses_with_domain"
      }
    }
  ]
}
```

**Miscellaneous Rough Notes**:

1. Trfm_ta_in_sb: has ids of students and their attempt ids, and other info.
    a. Has Load_ta_from_tw as dependency
    b. Give 2 things, but how to connect attempt_id to window_id?

2. Trfm_school_board_reports_long uses (gives "window_code", "uid", "item_domain", **“score”, “weight” — to be totaled**):
    a. uses Trfm_ta_detail_in_sb  (should contain all the attempts)
    b. **Trfm_responses_with_domain**
    c. The records are for each individual attempt.
3. Trfm_school_board_reports_scores_by_domain:
    a. Should give n_seen, n_correct as seen in “trfm_stu_perf_overall”
4. Furthermore, n_correct is not used in any other slug. So I am guessing it is in the same tables as n_seen

I interpret this task as — **for each item domain, group all the attempts which have the same item-domain and calculate their totals**.

**Somehow very similar to Trfm_school_board_reports_long implementation with group by and aggregate features.**

- Somewhere among the two:
    - **Weight:** used in Trfm_school_board_reports_long
- Trfm_ta_detail_in_sb
    - **Uid**
    - **Window_code**
- Trfm_responses_with_domain
    - Missing description, But clearly has domain info
    - It says that each test has items and item responses
    - So has the item_id  (which is connected to a domain) from the question bank register and the responses to each item id for each attempt.
    - Cols
        - Is_invalid
        - "ta_id",
        - "item_id"
        - "**score**",
        - "is_nr",
        - "is_invalid",
        - "formatted_response",
        - "formatted_response_strict",
        - "response_type",
        - "is_score_override"
        - "ta_id": "test_attempt_id",
        - "td_id": "test_design_id",
        - "weight": "score_max",
        - **Item_domain**
- Trfm_school_board_reports_scores_by_domain
    - Contains **n_seen** and **n_correct**
    - "test_window_id",
    - "tw_type_slug",
    - "window_code",
    - "uid",
    - "s_name",
    - "sc_name",
    - "s_gid",
    - "sc_gid",
    - "stu_gov_id",
    - "first_name",
    - "last_name"

## 5. Describe the process you would take to verify and update the `schema_simple` properties if you were provided all tables.

I would run the process described by “trfm_scores_by_domain”. This should create a dataframe. Then for each of the column names in this dataframe, I would check if it matches with one of the column names in the “schema_simple”. Furthermore, I can also check the datatype of each column in the created dataframe with the one I expect from the column names in the “schema_simple”. In this manner, the data could be validated so that it conforms to the initial design choices for “trfm_scores_by_domain”.

Since n_seen, n_correct are calculated from Trfm_school_board_reports_scores_by_domain dataframe, we need to make sure they are accessed properly and are stored in proper format in that table.